Tools: letting the model ask for an action

Instead of describing what to check, the model asks our code to check it. Our code decides whether to.

60 min | Recorded | Class 10 of 15

10

What this session covers

01
From talking to doing
2 slides

02
The loop
2 slides

03
Keeping it safe
3 slides

04
Code walkthrough
4 slides

© 2026 Devansh Parapalli

Section 1 of 4

From talking to doing

A model on its own can only produce text.

01

A tool is a function we describe to the model

© 2026 Devansh Parapalli

The model **asks**. Our code **acts**.

The model never touches InvoiceHub or Tracklane. It asks; our code runs the lookup and hands back the result.

© 2026 Devansh Parapalli

A tool is a function we describe to the model

Name, what it does, what it needs.

01 get_batch_status
Status of an InvoiceHub import batch, given an ID like IH-260615-11.

02 get_shipment
Where a shipment is and why, given LOL-SH-4472190.

03 create_ticket
Opens a ticket. A person has to say yes first.

© 2026 Devansh Parapalli

Section 2 of 4

The loop

Ask, call, return, repeat, stop.

01

Every tool call runs the same loop until the model has an answer

02

For “is the batch stuck?”, the model makes two calls and asks us once

© 2026 Devansh Parapalli

Every tool call runs the same loop until the model has an answer

1	2	3	4	5
Send the question and the tool list	The model asks for a tool and arguments	Our code checks and runs it	The result goes back to the model	Repeat, or stop with an answer

© 2026 Devansh Parapalli

For “is the batch stuck?”, the model makes two calls and asks us once

We asked: “Is batch IH-260615-11 stuck? If it is, open a ticket for AP systems.”

1. get_batch_status(“IH-260615-11”)
→ processing since 06:00, 14 invoices

2. create_ticket(“ap-systems”, “Batch stuck since 06:00”)
→ waits for a person to say yes

3. Answer: “It’s been processing for over 2 hours, which the runbook calls stuck. Ticket raised once you approved.”

© 2026 Devansh Parapalli

Section 3 of 4

Keeping it safe

Tools make mistakes real.

01

Some actions never run without a person, whatever the model says

02

Check every argument the model sends, the way you'd check a form from a stranger

03

Errors go back to the model as results, not as crashes

© 2026 Devansh Parapalli

Some actions never run without a person, whatever the model says

Change supplier bank details	The fraud path from the runbook. Two people, always.
Anything in PayGrid	Payroll errors reach people's bank accounts.
Release a customs hold or regenerate customs documents	Fines and legal exposure.
Grant access above Internal	Access reviews exist for a reason.

© 2026 Devansh Parapalli

Check every argument the model sends, the way you'd check a form from a stranger

- The batch ID matches `IH-\d{6}-\d{2}`, or the call fails
- The queue is one of ours, or the call fails
- Read-only tools first; anything that writes goes behind approval
- A step limit (5 here), so a confused model can't loop forever

© 2026 Devansh Parapalli

Arguments come from text the model read, and that text can come from a ticket. Class 7's injection example applies here with real consequences.

Errors go back to the model as results, not as crashes

Crash

- The run stops
- Nobody finds out why
- The user sees nothing

Result

- `{"error": "no such batch"}`
- The model can ask for the right ID
- Or tell the user plainly

© 2026 Devansh Parapalli

Section 4 of 4

Code walkthrough

Three fake tools and the loop.

01
A tool is an ordinary function plus a description the model can read

02
The loop is short: ask, run what it asked for, send the result back

03
Where today sits on the card

04
What to do next

© 2026 Devansh Parapalli

A tool is an ordinary function plus a description the model can read

code/class10/tools.py

```
def get_batch_status(batch_id: str) -> dict:
    return BATCHES.get(batch_id, {"error": "no such batch"})

def create_ticket(queue: str, summary: str) -> dict:
    if input(f"Create ticket in {queue}: {summary!r}? [y/N] ").lower() != "y":
        return {"created": False, "reason": "declined by operator"}
    return {"created": True, "id": "TKT-419001"}
```


© 2026 Devansh Parapalli

The loop is short: ask, run what it asked for, send the result back

code/class10/tools.py

```
for _ in range(max_steps):
    reply = client().chat.completions.create(model=MODEL, messages=messages, tools=SCHEMAS)
    msg = reply.choices[0].message
    if not msg.tool_calls: return msg.content
    messages.append(msg)
    for call in msg.tool_calls:
        result = TOOLS[call.function.name](**json.loads(call.function.arguments))
        messages.append({"role": "tool", "tool_call_id": call.id, "content":
json.dumps(result)})
```


© 2026 Devansh Parapalli

Where today sits on the card

1. A script	The steps are known and don't change.
2. A rule	The decision is known and must be explainable line by line.
3. Classical ML	Labelled history, and the input is a table or short text.
4. A small trained model	Lots of text, images or audio, enough labels, cost per call matters.
5. An LLM with a prompt	Varied language in and out, few labels, someone checks the output.
6. An LLM plus retrieval, tools or agents	**Needs private knowledge, has to act, or takes several steps.**
7. Not yet	No ground truth, needs a guarantee, or nobody owns the result.

© 2026 Devansh Parapalli

What to do next

1. Run `tools.py` and decline the ticket. See what the model says.
2. Add a `get_runbook` tool that uses the class 9 search.
3. Read the class 11 pre-reading on MCP.

Questions after class: the course channel.

© 2026 Devansh Parapalli